NESNE YÖNELİMLİ PROGRAMLAMA

Ders 7: Nesne Yönelimli Programlama IV

Konular

- Kalıtım (Inheritance)
  - Virtual
  - Abstract
  - Interface Polymorphism
  - Sealed

Inheritance (Kalıtım, Miras alma)

- Inheritance (miras alma, kalıtım), bir nesnenin özelliklerinin farklı nesneler tarafından da kullanılabilmesine olanak sağlayan NYP özelliğidir.
- Öncelikle geçen haftaki konumuzda gördüğümüz:
  - Türetme
  - Erişim Belirteçleri
  - Yapıcılar ve Kalıtım

  konularını hatırlayalım.

Nesne Atama

- C# tür güvenliğine maksimum derecede önem vermektedir. Bu yüzden, eğer özel tür dönüşüm operatörleri bildirilmemişse farklı sınıf türlerinden nesneler birbirlerine atanamaz.

```
public class X
{
    public int a = 0;
}

public class Y
{
    public int b = 0;
}

class Program
{
    static void Main(string[] args)
    {
        Y _y = new Y();
        X _x = new X();

        _y = _x;
    }
}
```

Atama işlemi yapılamaz.

Ana ve yavru sınıf nesneleri

- Ancak türeyen sınıflarda bu kural delinir. Ana sınıf türünden nesnelere yavru sınıf türünden nesneler atanabilir. Örneğin C#'taki temel veri türlerinin object sınıfından türetildiğini söylemiştik. Bu sayede bir object nesnesine her türden veri atanabilir.

```
public class X
{
    public int a = 0;
}

public class Y : X
{
    public int b = 0;
}

class Program
{
    static void Main(string[] args)
    {
        Y _y = new Y();
        X _x = new X();

        _x = _y;
    }
}
```

Atanabilir, bu sayede base'in durumu değiştirilebilir.

Bu programda gözden kaçırmamamız gereken nokta nesne1 üzerinden B sınıfının kendine özgü üye elemanlarına erişemeyeceğimizdir. Bu şekilde ana sınıf türünden bir nesnenin kullanılabildiği her yerde yavru sınıf türünden bir nesneyi de kullanabiliriz. Örneğin bu örneğimizi düşünecek olursak bir metodun parametresi A tipinden ise bu metoda parametre olarak B tipinden bir nesneyi de verebiliriz. Çünkü her B nesnesi A nesnesinin taşıdığı bütün üye elemanları taşır. Bunu bilinçsiz tür dönüşümüne benzetebiliriz.

object sınıfı

- Şimdiye kadar bütün temel veri türlerinin object sınıfından türediğini söylemiştim. Asıl bombayı şimdi patlatıyorum. Bütün sınıflar gizlice object sınıfından türer. Bunu kanıtlamak için şu programı yazın:

```
using System;
class A
{
}
class Ana
{
   static void Main()
   {
      A nesne=new A();
      Console.WriteLine(nesne.ToString());
   }
}
```

- ToString() metodu normalde object sınıfına aittir. Ancak bütün sınıf nesneleriyle kullanılabilir. Bu programda ekrana nesnenin türü olan A yazılır. Başka bir örnek:

```
using System;
class A
{
}
class Ana
{
   static void Main()
   {
      A nesne=new A();
      object a=nesne;
   }
}
```

toString

```
public class X
{
    public override string ToString()
    {
        return "Ne istersen göster";
    }
}


class Program
{
    static void Main(string[] args)
    {
        X _x = new X();
        Console.WriteLine(_x);
        Console.ReadKey();
    }
}
```

- ToString() metodu normalde object sınıfına ait olduğunu söylemiştik. Eğer kendinize özel bir ToString() tanımlamak isterseniz override yapıp yeni bir string değeri döndürebilirsiniz.
- Override, (Geçersiz kılma) eski fonksiyonu geçersiz kılıp yeni durumu uygular.

Türemiş sınıfta aynı metotu üretme

```
public class X
{
    public void test()
    {
        Console.WriteLine("Ben X");
    }
}

public class Y : X
{
    public void test()
    {
        Console.WriteLine("Ben Y");
    }
}

class Program
{
    static void Main(string[] args)
    {
        X _x = new X();
        _x.test();
        Y _y = new Y();
        _y.test();
        Console.ReadKey();
    }
}
```

- NYP yandaki örnek çok biçimliliktir (polimorfizm).
- Ancak, klasik bir sınıftan farklı olarak türemiş sınıfta base sınıfla aynı isme, aynı parametre sayısına ve veri türüne sahip bir metot tanımlayabiliriz.
- Kodumuz çalışır. X için X'in test metodunu, Y için Y'nin X metodunu çalıştırır. Ancak altı çizili hata verir.
- Olayın farkında olduğunuzu new yazıp berlitebilirsiniz. (Değişkenlerde benzer bir durum yapmıştık. – Geçen hafta)

Türemiş sınıfta aynı metotu üretme

```
public class X
{
    public void test()
    {
        Console.WriteLine("Ben X");
    }
}

public class Y : X
{
    new public void test()
    {
        Console.WriteLine("Ben Y");
    }
}

class Program
{
    static void Main(string[] args)
    {
        X _x = new X();
        _x.test();
        Y _y = new Y();
        _y.test();
        Console.ReadKey();
    }
}
```

- Yeşil hatalar yanlışlıkla hata yapabileceğiz anlamında karşınıza çıkar. Programın çalışmasını etkilemezler.
- New sayesinde olayın farkında olduğunuzu derleyiciye söylemiş olursunuz. Derleyici de sizi uyarmayı bırakır.

Virtual (Sanal) Metotlar

- Virtual methods (sanal metotlar), base class (temel sınıf) içinde bildirilmiş ve derived class (türemiş sınıf) içinde de tekrar bildirilebilen metotlardır. Böylelikle sanal metotlar kullanılarak NYP'de çok sık başvurulan çok biçimliliği (polimorphizm) uygulanmış olur. Yani temel sınıfta bir sanal metot bildirildiğinde bu temel sınıftan türeyen sınıflardaki metotlar override edilerek, temel sınıftaki sanal metotu devre dışı bırakabilirler.
- Sanal metotları bildirmek için virtual anahtar sözcüğü kullanılır. Bu anahtar sözcükle, sanal metotumuz bildirilmiş olur. Türeyen sınıfta ise, temel sınıftaki sanal metotları devre dışı bırakmak için override anahtar sözcüğü kullanılır. Yani base class'da virtual olarak tanımladığım bir metotu derived class içinde override edebilirim. Eğer override edersem derived class içindeki metot çalışır, eğer etmez isem direkt olarak base class içindeki virtual metot çalışır.
- Virtual metotlar private olarak tanımlanamazlar, public olmak zorundadırlar. Zaten private olmasının bir anlamı yok çünkü aksi halde derived class içinde bunları override edemeyiz. Public olmasının yanı sıra protected, internal şeklinde de bildirilebilirler.

Virtual Metotlar

```
public class Personel
{
    public string AdSoyad;
    protected decimal maas;
    public Personel(string _adsoyad, decimal _maas)
    {
        this.AdSoyad = _adsoyad;
        this.maas = _maas;
    }

    public virtual decimal UcretiHesapla()
    {
        return maas;
    }
}

public class Satici : Personel
{
    private decimal satisbonusu;
    public Satici(string _adsoyad, decimal _maas, decimal _satisbonusu)
        : base(_adsoyad, _maas)
    {
        this.satisbonusu = _satisbonusu;
    }

    public override decimal UcretiHesapla()
    {
        return maas + satisbonusu;
    }
}
```

Örneğimizi geçmiş konularla da birleştirdik.

Virtual bir metotta türemiş sınıf özgürdür. İster tanımlarsınız isterseniz tanımlamazsınız

Türemiş sınıfta tanımlama yaptığınızda override kullanımına dikkat.

Abstract (Soyut) Sınıf ve Metot

- Sınıflara ve metotlara uygulanan bir anahtar sözcük olduğu için iki bölümde inceleyelim.
- Abstract Class

Bir sınıfın abstract olmasındaki amaç, bir takım sınıflara taban sınıf olmaktır. Bunun anlamı abstract class'ların nesnesi üretilemez. Ayrıca, abstract bir sınıfın içerisinde abstract metotlar tanımlanabilir.

```
abstract class Personel
{

}
```

Araya yazılacak metotlar ya abstract olur ya da abstract olmayan metot ve değişkenleri Barındırabilir.

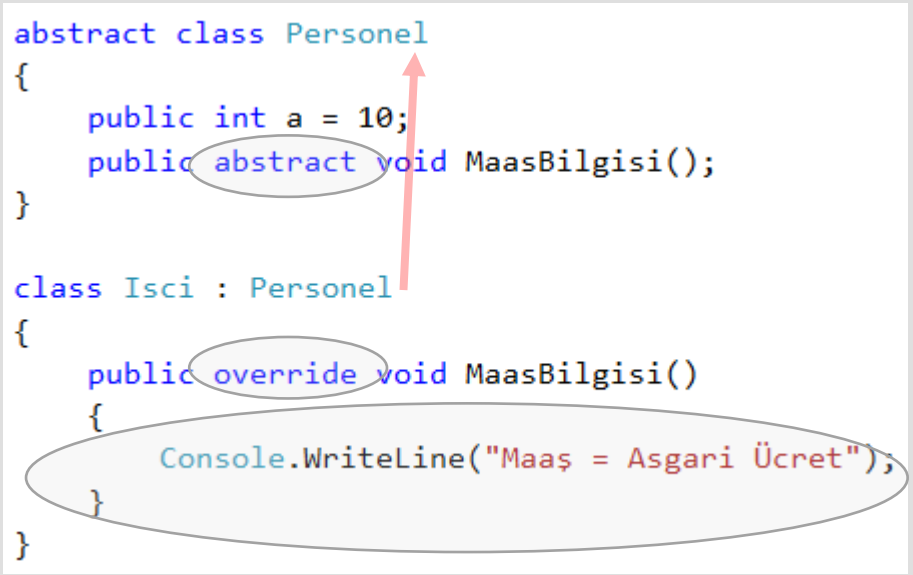
Abstract Metot

- Sadece abstract sınıfların içinde yazılabilen ve kalıtılan sınıflar tarafından override edilmek zorunda olan metotlardır.
- Private metotlara abstract anahtar sözcüğü uygulanamaz.
- Abstract metotları tanımlarken sadece imzalarını (geriye dönüş tipi ve parametreler) tanımlarız.
- Metodun gövdesi, kalıtılan sınıflarda oluşturulur. Diğer bir tanımlama şekli ise ; gövdesi olmayan ve ezilmek zorunda olan metotlardır.

```
abstract class Personel
{
    public int a = 10;
    public abstract void MaasBilgisi();
}
```

Abstract Metot Override Etme

- Abstract bir metot için override işlemi şarttır.
- Değişkenler abstract olarak tanımlanmaz.



```
abstract class Personel
{
    public int a = 10;
    public abstract void MaasBilgisi();
}

class Isci : Personel
{
    public override void MaasBilgisi()
    {
        Console.WriteLine("Maaş = Asgari Ücret");
    }
}
```

Abstract Property Override Etme

- Property abstract yapıldığında set veya get yapılıp yapılmayacağı abstract sınıfta bildirilir. Ancak sadece tanımlama yapılır. Örneğin Maas property'sine abtract sınıf içinde sadece get özelliği verilmiş. Bu durumda Isci sınıfındaki Maas property'sine sadece get tanımlanabilir ama set tanımlanamaz.

```
abstract class Personel
{
    public int a = 10;
    public abstract int Maas
    {
        get;
    }
}

class Isci : Personel
{
    public override int Maas
    {
        get { return 1000; }
    }
}
```

Interface (Arayuz)

- Bir Interface, başka sınıflar için bir rehberdir.
- Interface'ler içinde kod yazılmaz dediğimiz gibi sadece yön verir class'a ne yapacagını söyler.
- Bir class birden çok interface'i uygulayabilir ama class bir interface'yi kendisine uyguluyor ise kesinlikle tüm metotlarını ezmek zorundadır. Fakat bu ezme işleminde override kullanılmaz.
- Interface'leri tanımlarken default (geçerli) olarak public atanır protected veya private olarak belirtemeyiz.
- Interface'lerin yapıcı metotu olmaz aynı şekilde değişkenleri tanımlayamayız.
- Arayüz elemanlarını static olarak tanımlayamayız.

Interface

```
public interface IArayuz
{
    void EkranaYaz();

    int Yas
    {
        get;
        set;
    }

    string Isim
    {
        get;
        set;
    }
}
```

- I harfi Interface'e başlarken genelde kullanılır ve tavsiye edilir. .Net'in içindeki Interface'lerde bu standart olarak kabul edilmiştir.

```
public class Kisiler : IArayuz
{
    private int y;
    private string i;
    public Kisiler()
    {
        y = 18;
        i = "Yok";
    }
    public Kisiler(string ad, int yas)
    {
        Yas = yas;
        Isim = ad;
    }
    public int Yas
    {
        get{ return y; }
        set{
            if (value < 0) y = 0;
            else y = value;
        }
    }
    public string Isim
    {
        get{ return i; }

        set{ i = value; }
    }
    public void EkranaYaz()
    {
        Console.WriteLine("Adım:" + i);
        Console.WriteLine("Yaşım:" + y);
    }
}
```

Interface'in tüm üyeleri kullanılır. Ama sıralaması önemli değildir. Ayrıca türetilen sınıf içine interface'den farklı üyelerde yazılabilir.

Birden fazla interface

- Bir sınıf birden fazla interface içerebilir ve intarface'ler içindeki tüm üyeleri kullanmak zorundadır.

```
//İngiliz ölçü birileri için
interface IEnglishDimensions
{
    float Length();
    float Width();
}
//Diğer ölçü birimleri için
interface IMetricDimensions
{
    float Length();
    float Width();
}
```

```
class Box : IEnglishDimensions, IMetricDimensions
{
    float lengthInches;
    float widthInches;
    public Box(float length, float width)
    {
        lengthInches = length;
        widthInches = width;
    }
    float IEnglishDimensions.Length()
    {
        return lengthInches;
    }
    float IEnglishDimensions.Width()
    {
        return widthInches;
    }
    float IMetricDimensions.Length()
    {
        return lengthInches * 2.54f;
    }
    float IMetricDimensions.Width()
    {
        return widthInches * 2.54f;
    }
}
```

Birden fazla interface

```
class Box : IEnglishDimensions, IMetricDimensions
{
    float lengthInches;
    float widthInches;
    public Box(float length, float width)
    {
        lengthInches = length;
        widthInches = width;
    }
    float IEnglishDimensions.Length()
    {
        return lengthInches;
    }
    float IEnglishDimensions.Width()
    {
        return widthInches;
    }
    float IMetricDimensions.Length()
    {
        return lengthInches * 2.54f;
    }
    float IMetricDimensions.Width()
    {
        return widthInches * 2.54f;
    }
}
```

```
//İngiliz ölçü birileri için
interface IEnglishDimensions
{
    float Length();
    float Width();
}
//Diğer ölçü birimleri için
interface IMetricDimensions
{
    float Length();
    float Width();
}
```

```
static void Main()
{
    Box box1 = new Box(30.0f, 20.0f);
    IEnglishDimensions eDimensions = (IEnglishDimensions)box1;
    IMetricDimensions mDimensions = (IMetricDimensions)box1;

    System.Console.WriteLine("Length(in): {0}", eDimensions.Length());
    System.Console.WriteLine("Width (in): {0}", eDimensions.Width());
    System.Console.WriteLine("Length(cm): {0}", mDimensions.Length());
    System.Console.WriteLine("Width (cm): {0}", mDimensions.Width());
}
```

Interface türüne göre sınıfa bilinçli dönüşüm yapılır.
Bilinçli dönüşüm sonuçları dönüşüm türünden değişkenlere aktarılabilir.

Multiple Inheritance (Çoklu Kalıtım)

- Bir sınıf direkt iki sınıfı kalıtım olarak kullanmaz. Dolaylı yoldan kullanabilir.
- Diğer taraftan interface örneklerinde olduğu gibi bu durum interface için geçerli değildir.

```
class Baba
{

}
class Anne: Baba
{

}
class Cocuk: Anne
{

}
```

```
class Baba
{

}
class Anne
{

}
class Cocuk: Anne, Baba
{

}
```

Error List

1 Error | 0 Warnings | 0 Messages

	Description
1	Class 'Cocuk' cannot have multiple base classes: 'Anne' and 'Baba'

```
interface Baba
{

}
interface Anne
{

}
class Cocuk: Anne, Baba
{

}
```

.Net Interface'leri

- .Net içinde bir çok interface türü vardır. Örneğin
  - IEnumerable
  - IList
  - IDictionary
  - IComparable

.Net Interface - IEnumerable

- IEnumerable
  - Tüm array'leri ve List'leri IEnumerable arayüzünden türetilmişlerdir. Bu sebepten dolayı IEnumerable üzerinden bu sınıf türlerine ulaşmak mümkündür.

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main()
    {
        int[] values = { 1, 2, 3 };
        List<int> values2 = new List<int>() { 1, 2, 3 };

        Display(values);
        Display(values2);
    }

    static void Display(IEnumerable<int> values)
    {
        foreach (int value in values)
        {
            Console.WriteLine(value);
        }
    }
}
```

Yandaki örnek IEnumerable arayüzü sayesinde sadece tek fonksiyon kullanarak hem int[] hem de List<int> veri türlerini ekranda gösterir.

Bu örnekte int[] ve List veri türlerinin IEnumerable özelliğini içermesini kullandık.
IComparable örneğinde kendi sınıfımıza türetme yapacağız.

.Net Interface - IComparable

- Karşılaştırma işlemi ile dizi üzerinde sıralama yapmak için kullanılır. Bu arayüz tanımlandığında CompareTo metodunu kullanmak şarttır.

```
using System;
using System.Collections.Generic;

class Calisan : IComparable<Calisan>
{
    public int Maas { get; set; }
    public string Ad { get; set; }

    public int CompareTo(Calisan diger_Calisan)
    {
        //Maas eşit ise alfabetik sıralama [A'dan Z'ye]
        if (this.Maas == diger_Calisan.Maas)
        {
            return this.Ad.CompareTo(diger_Calisan.Ad);
        }
        // Geçerli durum if gerçekleşmezse çalışır. [Yüksekten düşüğe]
        return diger_Calisan.Maas.CompareTo(this.Maas);
    }

    //toString metodu direkt object içinde vardır ve virtual türündendir.
    //Override edilmesi şarttır
    public override string ToString()
    {
        return this.Maas.ToString() + "," + this.Ad;
    }
}
```

Arayüz

Bu metot şart kullanılmalı. Arayüzden geliyor.

Object'ten geliyor, override edilip toString durumunda ne değer döndüreceği söyleniyor.

.Net Interface - IComparable

```cs
using System;
using System.Collections.Generic;

class Calisan : IComparable<Calisan>
{
    public int Maas { get; set; }
    public string Ad { get; set; }

    public int CompareTo(Calisan diger_Calisan)
    {
        //Maas eşit ise alfabetik sıralama [A'dan Z'ye]
        if (this.Maas == diger_Calisan.Maas)
        {
            return this.Ad.CompareTo(diger_Calisan.Ad);
        }
        // Geçerli durum if gerçekleşmezse çalışır. [Yüksekten düşüğe]
        return diger_Calisan.Maas.CompareTo(this.Maas);
    }

    //toString metodu direkt object içinde vardır ve virtual türündendir.
    //Override edilmesi şarttır
    public override string ToString()
    {
        return this.Maas.ToString() + "," + this.Ad;
    }
}
```

```cs
class Program
{
    static void Main()
    {
        List<Calisan> list = new List<Calisan>();
        list.Add(new Calisan() { Ad = "Ali", Maas = 10000 });
        list.Add(new Calisan() { Ad = "Veli", Maas = 10000 });
        list.Add(new Calisan() { Ad = "Ayşe", Maas = 10000 });
        list.Add(new Calisan() { Ad = "Fatma", Maas = 500000 });
        list.Add(new Calisan() { Ad = "Metin", Maas = 8000 });

        //IComparable.CompareTo() Fonksiyonuna göre sıralar
        list.Sort();

        //Calisan.ToString kullanır
        foreach (var element in list)
        {
            Console.WriteLine(element);
        }
    }
}
```

Sealed

- sealed, sınıflara ve üyelere(metot ve property) uygulanabilen bir anahtar sözcüktür.
- Anlamı; kapalı, mühürlü şeklinde çevrilebilir.
- Sınıflar için kalıtımı, üyeler için ise override edilmeyi önler.
- Sınıflar üzerine sealed işlemi

```
public class A
{

}

public class B : A
{

}

public class C : B
{

}
```

C sınıfı sealed yaptığımızda C artık türetilemez.

```
public class A
{

}

public class B : A
{

}

sealed public class C : B
{

}

public class D : C
{

}
```

Error List

1 Error | 0 Warnings

	Description
1	'D': cannot derive from sealed type 'C'

Sealed

- Sealed metot ve property'e de uygulanabilir.

```
class X
{
    protected virtual void F() { Console.WriteLine("X.F"); }
    protected virtual void F2() { Console.WriteLine("X.F2"); }
}
class Y : X
{
    sealed protected override void F() { Console.WriteLine("Y.F"); }
    protected override void F2() { Console.WriteLine("Y.F2"); }
}
class Z : Y
{
    //Y sınıfındaki metot sealed olduğu için Override edilemez.
    //protected override void F() { Console.WriteLine("C.F"); }

    //Override edilebilir.
    protected override void F2() { Console.WriteLine("Z.F2"); }
}
```

Sealed

- .Net Framework içerisinden örnek vermek gerekirse; string tipi de sealed olarak işaretlenmiştir. Yani string tipinden kalıtım yapamayız.
- string üzerindeyken sağ tuş Goto Definition diyelim

```
Assembly mscorlib.dll, v4.0.0.0

using System.Collections;
using System.Collections.Generic;
using System.Globalization;
using System.Reflection;
using System.Runtime;
using System.Runtime.ConstrainedExecution;
using System.Runtime.InteropServices;
using System.Security;
using System.Text;

namespace System
{
    public sealed class String : IComparable, ICloneable, IConvertible, IComparable<string>, IEnumerable<char>, IEnumerable, IEquatable<string>
    {
        public static readonly string Empty;

        public String(char* value);
        public String(char[] value);
        public String(sbyte* value);
        public String(char c, int count);
        public String(char* value, int startIndex, int length);
        public String(char[] value, int startIndex, int length);
        public String(sbyte* value, int startIndex, int length);
        public String(sbyte* value, int startIndex, int length, Encoding enc);
```

NYP Hatırlatma

- İyi şekilde NYP öğrendiğinizde .Net'in veya ileride karşılaşabileceğiniz diğer NYP tabanlı programlama dillerinin yapısını çok rahat çözebilirsiniz. Özellikle Goto Defition çok işinize yarar. Örneğin ilk defa Random özelliği kullanacaksınız. Goto Defition yapalım.

```
Assembly mscorlib.dll, v4.0.0.0

using System.Runtime;
using System.Runtime.InteropServices;

namespace System
{
    public class Random
    {
        public Random();
        public Random(int Seed);

        public virtual int Next();
        public virtual int Next(int maxValue);
        public virtual int Next(int minValue, int maxValue);
        public virtual void NextBytes(byte[] buffer);
        public virtual double NextDouble();
        protected virtual double Sample();
    }
}
```

Bazı Çıkarımlar:

- System NameSpace'i içindedir.
- Static sınıf değil, şart new ile yaratılmalı.
- İki constructor'ı var.
- 5 tane override edilebilir virtual metodu vardır.
- Bir tane sadece türetilen sınıf tarafından override edilebilir bir protected belirteçli metodu var.

Burada kod yoktur, sadece sınıf hakkında bilgi edinebilirsiniz.

NYP Hatırlatma

```
namespace System
{
    public struct DateTime : IComparable, IFormattable, IConvertible, ISerializable, IComparable<DateTime>, IEquatable<DateTime>
    {
        public static readonly DateTime MaxValue;
        public static readonly DateTime MinValue;

        public DateTime(long ticks);
        public DateTime(long ticks, DateTimeKind kind);
        public DateTime(int year, int month, int day);
        public DateTime(int year, int month, int day, Calendar calendar);
        public DateTime(int year, int month, int day, int hour, int minute, int second);
        public DateTime(int year, int month, int day, int hour, int minute, int second, Calendar calendar);
        public DateTime(int year, int month, int day, int hour, int minute, int second, DateTimeKind kind);
        public DateTime(int year, int month, int day, int hour, int minute, int second, int millisecond);
        public DateTime(int year, int month, int day, int hour, int minute, int second, int millisecond, Calendar calendar);
        public DateTime(int year, int month, int day, int hour, int minute, int second, int millisecond, DateTimeKind kind);
        public DateTime(int year, int month, int day, int hour, int minute, int second, int millisecond, Calendar calendar, DateTimeKind kind);

        public static TimeSpan operator -(DateTime d1, DateTime d2);
        public static DateTime operator -(DateTime d, TimeSpan t);
        public static bool operator !=(DateTime d1, DateTime d2);
        public static DateTime operator +(DateTime d, TimeSpan t);
        public static bool operator <(DateTime t1, DateTime t2);
        public static bool operator <=(DateTime t1, DateTime t2);
        public static bool operator ==(DateTime d1, DateTime d2);
        public static bool operator >(DateTime t1, DateTime t2);
        public static bool operator >=(DateTime t1, DateTime t2);
```

- İşaretlenmiş bölümlerin tümü derslerde anlatıldı.
- Ayrıca, bu konulara çalışırken tüm konuları içeren bol yorum satırlı uygulamayı da incelemeyi unutmayın.

Kaynaklar

- https://tr.wikibooks.org/wiki/C_Sharp_Programlama_Dili/Kal%C4%B1t%C4%B1m
- http://www.ismailgursoy.com.tr/virtual-methods-sanal-metotlar/
- http://onursalkaya.blogspot.com.tr/2011/03/c-abstract-nedir.html
- http://www.gokhanakgol.com/2011/11/10/c-ile-interface-kullanimi/
- http://onursalkaya.blogspot.com.tr/2011/12/c-sealed-anahtar-sozcugu.html